DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: Helios Watch Store E-Commerce Database

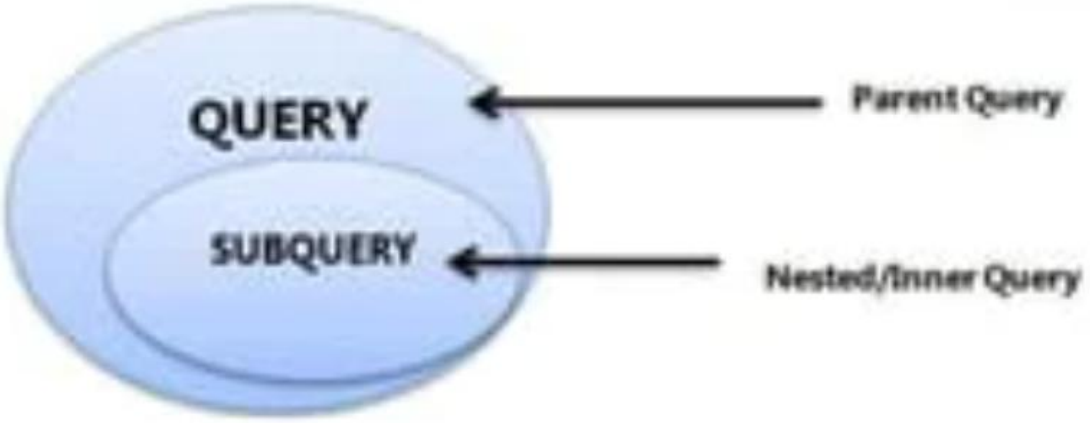
Skill Task: DBMS

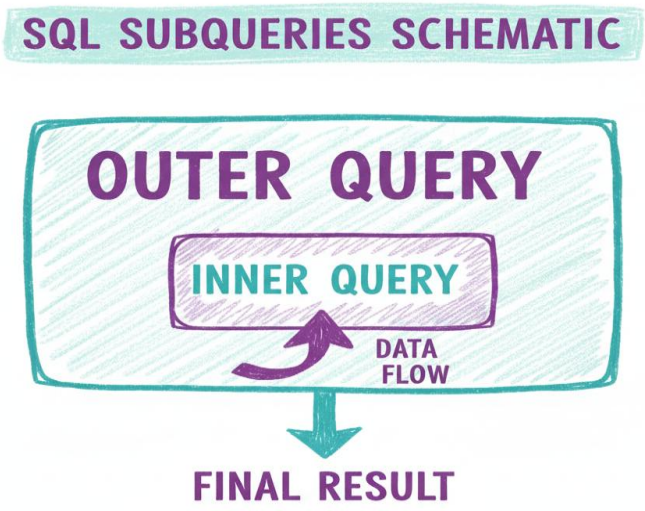
Week: 10

1.Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.





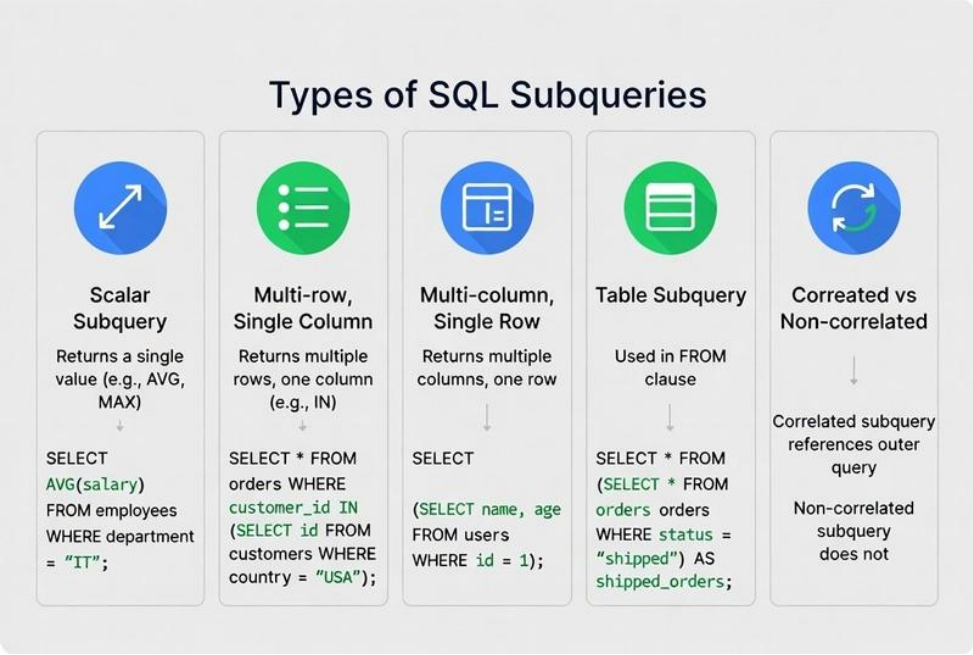
Basic Structure:

```sql
SELECT
    order_id,
    order_date,
    customer_id
FROM
    sales.orders
WHERE
    customer_id IN (
        SELECT
            customer_id
        FROM
            sales.customers
        WHERE
            city = 'New York'
    )
ORDER BY
    order_date DESC;
```

outer query

subquery

Types of SQL Subqueries:



FIND ORDERS ABOVE AVERAGE ORDER AMOUNT

```
SQL> SELECT Order_ID,
  2  Customer_ID,
  3  Order_Date,
  4  Total_Amount
  5  FROM Orders
  6  WHERE Total_Amount >
  7  (SELECT AVG(Total_Amount)
  8  FROM Orders)
  9  ORDER BY Total_Amount DESC;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
---------- ----------- --------- ------------
      1008           8 08-SEP-26        12999
      1005           5 05-SEP-26         7499
      1001           1 15-SEP-26         6999
      1004           4 04-SEP-26         5999
```

Explanation:

The inner query calculates the average order amount from the `Orders` table using the `AVG()` aggregate function. The outer query retrieves the details of orders whose `Total_Amount` is greater than the calculated average. Thus, this query identifies all orders with an amount above the average order value.

2. FIND PRODUCTS ABOVE AVERAGE PRICE

```
SQL>
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price >
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       104
Smart Fit Pro
      5999

       105
Smart Active Watch
      7499

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       108
Royal Gold Edition
     12999
```

Explanation:

The inner query calculates the average price of all products in the `Product` table using the `AVG()` aggregate function. The outer query retrieves the `Product_ID`, `Product_Name`, and `Price` of products whose price is greater than the calculated average price. Thus, this query identifies all products priced above the average product price.

2.1 PRODUCTS BELOW AVERAGE PRICE

```
SQL> SELECT Product_ID,
  2         Product_Name,
  3         Price
  4  FROM Product
  5  WHERE Price <
  6        (SELECT AVG(Price)
  7         FROM Product)
  8  ORDER BY Price ASC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
       103
Digital Sport Watch
      1999

       101
Classic Silver Watch
      2799

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
```

```
       102
Elegant Gold Watch
      4299

       106
Sports Chronograph

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
     PRICE
----------
      4599

       107
Outdoor Pro Watch
      5299
```

Explanation:

The inner query calculates the average price of all products in the `Product` table using the `AVG()` aggregate function. The outer query retrieves the `Product_ID`, `Product_Name`, and `Price` of products whose price is less than the calculated average price. Thus, this query identifies all products priced below the average product price.

3. IDENTIFY CUSTOMERS WITH MAXIMUM PURCHASES

3.1 CALCULATE CUSTOMER-WISE PURCHASE

```
SQL> SELECT Customer_ID,
  2         SUM(Total_Amount) AS Total_Purchase
  3  FROM Orders
  4  GROUP BY Customer_ID
  5  ORDER BY Total_Purchase DESC;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          8          12999
          5           7499
          1           6999
          4           5999
          7           5299
          6           4599
          2           3999
          9           2499
          3           1999

9 rows selected.
```

Explanation:

The `SUM()` aggregate function calculates the total purchase amount for each customer from the `Orders` table. The `GROUP BY Customer_ID` clause groups the orders based on each customer's ID. The `ORDER BY Total_Purchase DESC` clause displays the customers in descending order of their total purchase amount. Thus, this query calculates and displays the total purchase made by each customer.

3.2 FIND THE MAXIMUM PURCHASE AMOUNT

```
SQL> SELECT MAX(Total_Purchase) AS Maximum_Purchase
  2  FROM
  3  (
  4      SELECT Customer_ID,
  5             SUM(Total_Amount) AS Total_Purchase
  6      FROM Orders
  7      GROUP BY Customer_ID
  8  );

MAXIMUM_PURCHASE
----------------
           12999
```

Explanation:

The inner query calculates the total purchase amount for each customer using the `SUM()` function and groups the results by `Customer_ID`. The outer query uses the `MAX()` aggregate function to identify the highest value among those customer-wise purchase amounts. Thus, this query finds the maximum total purchase amount among all customers.

3.3 DISPLAY CUSTOMER DETAILS WITH MAXIMUM PURCHASE

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name,
  3         c.Last_Name,
  4         SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer c
  6  INNER JOIN Orders o
  7  ON c.Customer_ID = o.Customer_ID
  8  GROUP BY c.Customer_ID,
  9           c.First_Name,
 10           c.Last_Name
 11  HAVING SUM(o.Total_Amount) =
 12         (
 13             SELECT MAX(Total_Purchase)
 14             FROM
 15             (
 16                 SELECT SUM(Total_Amount) AS Total_Purchase
 17                 FROM Orders
 18                 GROUP BY Customer_ID
 19             )
 20         );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
          8 Swetha
Balaji                                                      12999
```

Explanation:

The query joins the `Customer` and `Orders` tables using the `Customer_ID` column to retrieve customer details along with their orders. The `SUM()` function calculates the total purchase amount for each customer. The `GROUP BY` clause groups the records customer-wise, while the `HAVING` clause compares each customer's total purchase with the maximum purchase amount calculated by the inner query. Thus, this query displays the details of the customer who has made the highest total purchase.

4. COMPLEX BUSINESS QUERIES

4.1 CUSTOMERS WHO PLACED ORDERS ABOVE AVERAGE ORDER VALUE

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name,
  3         c.Last_Name,
  4         o.Order_ID,
  5         o.Total_Amount
  6  FROM Customer c
  7  INNER JOIN Orders o
  8  ON c.Customer_ID = o.Customer_ID
  9  WHERE o.Total_Amount >
 10        (
 11            SELECT AVG(Total_Amount)
 12            FROM Orders
 13        )
 14  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
          8 Swetha
Balaji                                               1008        12999

          5 Keerthana
Mohan                                                1005         7499

          1 Renu
Srinivasan                                           1001         6999


CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
          4 Rahul
Sharma                                               1004         5999
```

Explanation:

The inner query calculates the average order amount from the `Orders` table using the `AVG()` aggregate function. The outer query joins the `Customer` and `Orders` tables using the `Customer_ID` column and retrieves the customer details and order amounts for orders exceeding the calculated average. Thus, this query identifies customers who placed orders above the average order value.

5. FIND PRODUCTS WHOSE SALES ARE ABOVE AVERAGE

```
SQL> SELECT p.Product_ID,
  2         p.Product_Name,
  3         SUM(od.Subtotal) AS Total_Sales
  4  FROM Product p
  5  INNER JOIN Order_Details od
  6  ON p.Product_ID = od.Product_ID
  7  GROUP BY p.Product_ID,
  8           p.Product_Name
  9  HAVING SUM(od.Subtotal) >
 10         (
 11             SELECT AVG(Product_Sales)
 12             FROM
 13             (
 14                 SELECT SUM(Subtotal) AS Product_Sales
 15                 FROM Order_Details
 16                 GROUP BY Product_ID
 17             )
 18         )
 19  ORDER BY Total_Sales DESC;
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       108
Royal Gold Edition
      12999

       103
Digital Sport Watch
       7996

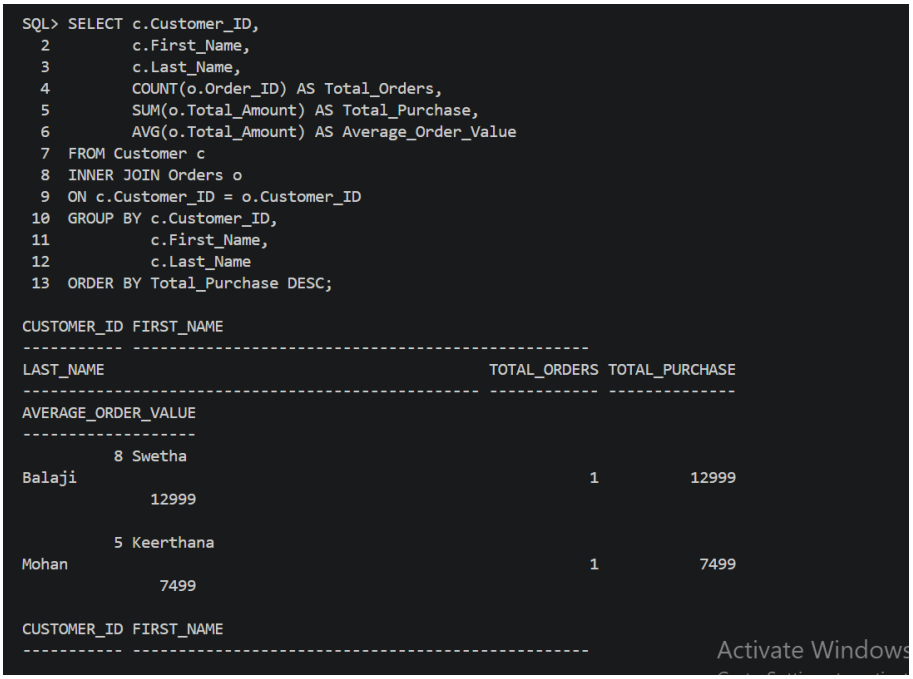
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------

       105
Smart Active Watch
       7499
```

Explanation:

The inner query calculates the total sales amount for each product using the `SUM()` function and groups the results by `Product_ID`. The next level of the inner query calculates the average of these product-wise sales amounts using the `AVG()` function. The outer query joins the `Product` and `Order_Details` tables, calculates the total sales for each product, and uses the `HAVING` clause to display products whose total sales are greater than the average product sales. Thus, this query identifies products with above-average sales.

6. CUSTOMER PURCHASE REPORT

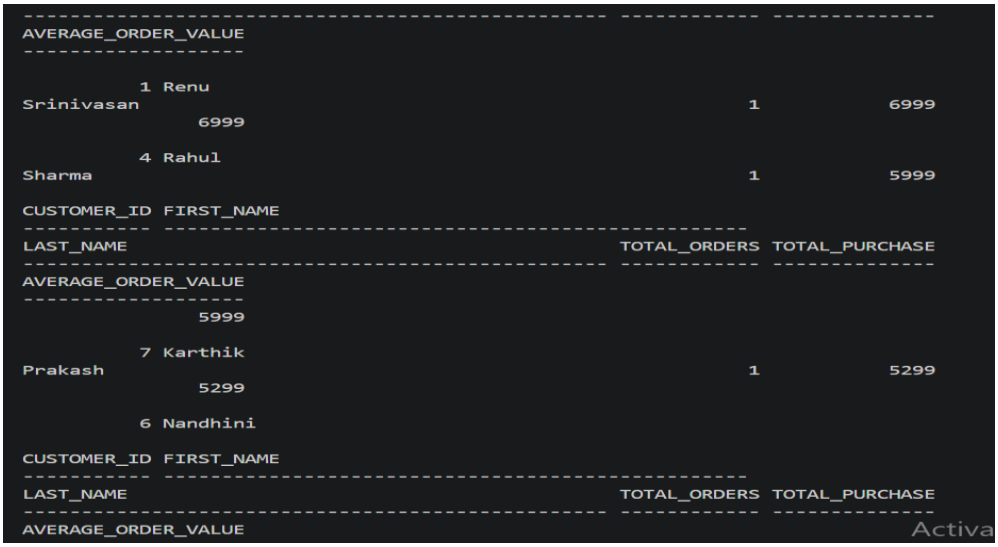


```
SQL> SELECT c.Customer_ID,
  2         c.First_Name,
  3         c.Last_Name,
  4         COUNT(o.Order_ID) AS Total_Orders,
  5         SUM(o.Total_Amount) AS Total_Purchase,
  6         AVG(o.Total_Amount) AS Average_Order_Value
  7  FROM Customer c
  8  INNER JOIN Orders o
  9  ON c.Customer_ID = o.Customer_ID
 10  GROUP BY c.Customer_ID,
 11           c.First_Name,
 12           c.Last_Name
 13  ORDER BY Total_Purchase DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------
         8 Swetha
Balaji                                                        1          12999
             12999

         5 Keerthana
Mohan                                                         1           7499
              7499

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
```



```
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------

           1 Renu
Srinivasan                                                    1           6999
                6999

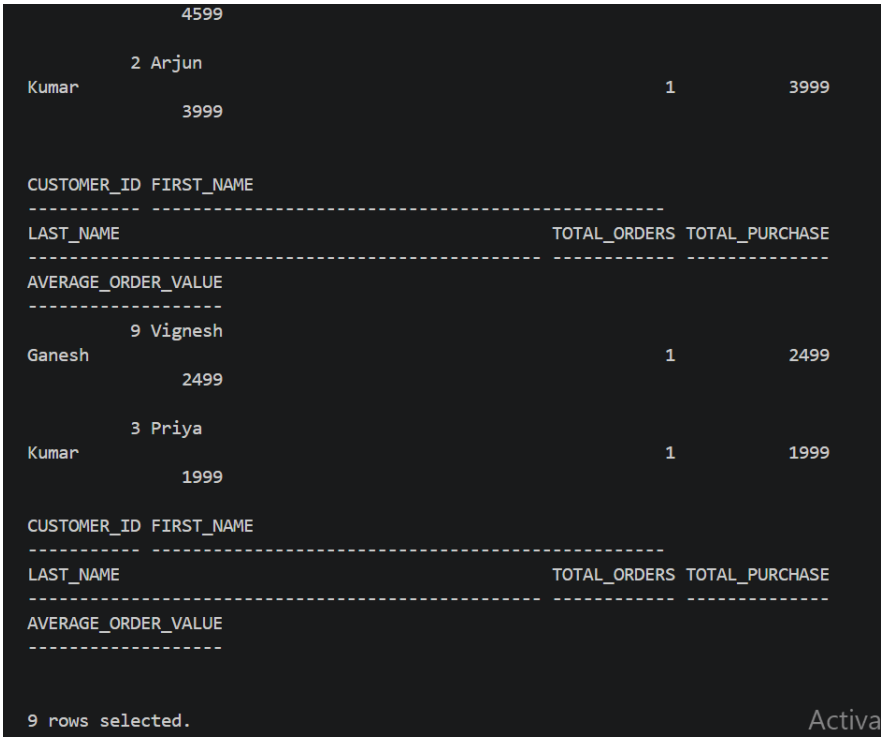
           4 Rahul
Sharma                                                        1           5999

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------
                5999

           7 Karthik
Prakash                                                       1           5299
                5299

           6 Nandhini

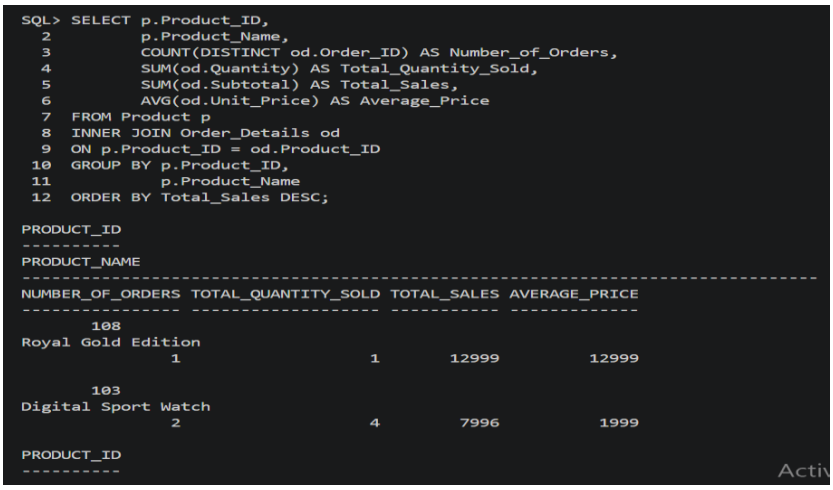
CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
```



Explanation:

The query joins the `Customer` and `Orders` tables using the `Customer_ID` column to retrieve customer purchase details. The `COUNT()` function calculates the total number of orders placed by each customer, while the `SUM()` function calculates their total purchase amount. The `AVG()` function calculates the average order value for each customer. The `GROUP BY` clause groups the results customer-wise, and the `ORDER BY` clause arranges the customers in descending order of their total purchase amount. Thus, this query generates a detailed customer purchase report.

7. ADVANCED PRODUCT SALES REPORT



```
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------

       105
Smart Active Watch
               1                   1        7499          7499

       104
Smart Fit Pro

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------
               1                   1        5999          5999

       107
Outdoor Pro Watch
               1                   1        5299          5299

       101

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
```

Activate Wi

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------
Classic Silver Watch
               2                   2        4998          2499

       106
Sports Chronograph
               1                   1        4599          4599


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------
       102
Elegant Gold Watch
               1                   1        3999          3999


8 rows selected.
```

Explanation:

The query joins the `Product` and `Order_Details` tables using the `Product_ID` column to retrieve product sales information. The `COUNT(DISTINCT od.Order_ID)` function calculates the number of unique orders containing each product. The `SUM()` function calculates the total quantity sold and total sales amount, while the `AVG()` function calculates the average selling price of each product. The `GROUP BY` clause groups the results product-wise, and the `ORDER BY` clause arranges the products in descending order of total sales. Thus, this query generates a detailed product sales report.

8. CATEGORY-WISE ADVANCED SALES REPORT

```
SQL> SELECT cat.Category_ID,
  2         COUNT(DISTINCT od.Order_ID) AS Total_Orders,
  3         SUM(od.Quantity) AS Total_Quantity_Sold,
  4         SUM(od.Subtotal) AS Total_Sales,
  5         AVG(od.Unit_Price) AS Average_Price
  6  FROM Category cat
  7  INNER JOIN Product p
  8  ON cat.Category_ID = p.Category_ID
  9  INNER JOIN Order_Details od
 10  ON p.Product_ID = od.Product_ID
 11  GROUP BY cat.Category_ID
 12  ORDER BY Total_Sales DESC;

CATEGORY_ID TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
----------- ------------ ------------------- ----------- -------------
          3            2                   2       13498          6749
          5            1                   1       12999         12999
          4            2                   2        9898          4949
          1            3                   3        8997          2999
          2            2                   4        7996          1999
```

Explanation:

The query joins the `Category`, `Product`, and `Order_Details` tables using the `Category_ID` and `Product_ID` columns to retrieve category-wise sales information. The `COUNT(DISTINCT od.Order_ID)` function calculates the total number of unique orders in each category. The `SUM()` function calculates the total quantity sold and total sales amount, while the `AVG()` function calculates the average selling price. The `GROUP BY` clause groups the results category-wise, and the `ORDER BY` clause arranges the categories in descending order of total sales. Thus, this query generates a detailed category-wise sales report.